

Received 17 December 2023, accepted 30 December 2023, date of publication 5 January 2024,
date of current version 11 January 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3350175

RESEARCH ARTICLE

Wireless Spy Camera Spotter System With Real-Time Traffic Similarity Analysis and WiFi Signal Tracing

HYEYOUNG AN^{ID}, WOOJIN PARK^{ID}, (Student Member, IEEE),
AND SOOCHANG PARK^{ID}, (Member, IEEE)

Department of Computer Engineering, Chungbuk National University, Cheongju 28644, Republic of Korea

Corresponding author: Soochang Park (cewinter@chungbuk.ac.kr)

ABSTRACT This paper introduces a novel method to detect WiFi-enabled spy cameras potentially invading user privacy via wireless streaming within a user's vicinity. So far, there have been social initiatives to conduct spy camera inspections to address this issue, and academic societies have been working on various detection methods. However, such studies typically rely on additional specific efforts, such as using specialized equipment to gather channel state information (CSI) or collecting extensive long-term data per location for machine learning approaches. In addition, some ideas could not recognize features of spy camera data due to data encryption. Hence, this paper comes up with a novel scheme for detecting wireless spy cameras based only on off-the-shelf devices. To achieve these functionalities, the proposed scheme consists of three phases: 1) classification, 2) detection, and 3) localization. First, in the classification phase, the collected packet headers are analyzed to identify and classify the upstream streaming protocols. In addition, the proposed scheme fulfills the detection phase that exploits the Nilsimsa algorithm to analyze packet similarity, identifying potential spy cameras by detecting consistent high similarity patterns in the classified data traffic. Finally, the proposed approach provides the localization phase that captures and analyzes Received Signal Strength Indication (RSSI) values at the target location and spots the camera position. Unlike existing methods, the proposed scheme does not rely on extensive data collection per location and complex mechanisms to interpret encrypted data. Moreover, compared to machine learning (ML)-based positioning means, which require significant data sets and long-term computation time, the scheme achieves real-time detection with minimal computational demands.

INDEX TERMS Spy camera, Nilsimsa, WiFi received signal strength indication (RSSI), traffic analysis, packet classification.

I. INTRODUCTION

In recent years, the field of surveillance technology has made remarkable advancements. These technological strides have transformed IP cameras, which are commonly employed for surveillance purposes, into more compact, cost-effective, and user-friendly devices. Moreover, the adoption of high-resolution capabilities, along with the integration of wireless and internet connectivity, has significantly contributed to their widespread popularity [1].

The associate editor coordinating the review of this manuscript and approving it for publication was Petros Nicopolitidis^{ID}.

IP cameras are digital video cameras strategically positioned in various environments to monitor and record activities [1], [2], [3]. These cameras can transmit their recorded footage over the internet, enabling remote access. This live video stream can be accessed concurrently from multiple devices like PCs, smartphones, tablets, and laptops, providing convenient monitoring options [2]. Consequently, IP cameras have gained widespread utilization due to increasing security concerns and the imperative to monitor the activities of individuals such as parents, children, and pets.

Despite their benefits, IP cameras can be easily abused for malicious purposes [1]. Spy cameras are cameras installed

with the intention of secretly capturing someone's private moments, such as in bathrooms, changing rooms, or hotel rooms. This issue, commonly known as "molka" in Korea, has become widespread and even earned its own name.¹ The majority of victims are women, and the captured videos are frequently sold on various pornographic websites. To address this issue, in September 2018, the Seoul Metropolitan Government formed a team of 8,000 members to inspect public restrooms in Seoul. Additionally, the police regularly conduct cleaning operations in public restrooms and hotels to discover and remove hidden cameras.² Most women are extremely concerned about spy cameras. However, as technology advances, cameras have become smaller, lighter, and more powerful. As a result, spy cameras have become increasingly difficult to detect as they are cleverly concealed in unexpected ways.

Consequently, various efforts are being made to detect these spy cameras. Although commercially available hardware equipment for detecting spy cameras exists, it often proves cumbersome and impractical for portability. Furthermore, these devices have the limitation of being able to detect only specific types of streams [1], [4]. There are also proposals to employ channel state information (CSI) analysis methods for detecting spy cameras. However, the CSI-based approach may not effectively detect spy cameras when there is no user movement. Furthermore, this method requires specific devices with the capability to extract CSI [1], [4], [5]. In addition, ML techniques can be employed to detect spy cameras. One approach involves comparing the recorded video to streaming network traffic and training a neural network to distinguish between normal and potentially suspicious camera signals [6]. Another method involves analyzing motion trajectories within a designated motion detection area to calculate the position of a camera [7]. However, the successful implementation of these techniques requires extensive data collection to effectively train the neural network [6]. As an alternative approach, there are available methods for analyzing streaming networks [8], [9], [10], [11], [12]. These techniques rely on examining the network's fluctuations, specifically in response to human movement. Consequently, spy cameras can be detected only when the requisite actions to trigger the solution are present.

This paper introduces an innovative approach for identifying WiFi-enabled spy cameras that are potentially streaming within a user's vicinity. The proposed scheme consists of three phases. In the classification phase, we record nearby data transmissions and categorize them based on the source and destination. Following this, the detection phase employs the Nilsimsa algorithm to analyze transmitted packets, disregarding encryption, protocol, or size constraints. The detection of high-similarity patterns suggests the presence of potential spy cameras. Finally, in the localization phase,

¹<https://www.abc.net.au/news/2021-06-17/spy-cam-sex-crimes-in-south-korea-leave-women-fearful/100214532>

²<https://koreabyeme.com/koreas-spycam-plague-how-to-spot-hidden-cameras/>

TABLE 1. Comparison of techniques used in spy camera detection.

Method of traffic analysis	Solution name	Traffic class.	Additional Data		Spy cam local.	Ref
			Human action	Other data		
CSI	CSI:DeSpy	✓	✓	✗	✗	[1]
	Detection and Localization	✓	✓	✓	✓	[5]
	LENSER	✓	✓	✓	✓	[13]
CSI+ML	DeepDeSpy	✓	✓	✗	✗	[4]
ML+other	LAPD	✗	✓	✓	✓	[14]
	Adversarial Localization	✗	✗	✓	✓	[15]
ML + Traffic pattern	Similarity of Simultaneous Observation	✓	✗	✓	✗	[6]
	DeWiCam	✓	✓	✗	✗	[8], [9]
	Motion Compass	✓	✓	✗	✓	[7]
	FindSpy	✓	✗	✗	✗	[16]
Traffic pattern	SCamF	✓	✓	✗	✓	[10]
	Blick and Flicker	✓	✓	✗	✗	[11]
	SnoopDog	✗	✓	✗	✓	[12]

Legend: class-classification, local-localization, Ref-References.

we estimate the approximate position of the spy camera by leveraging the RSSI value associated with the MAC (Media Access Control) address of the detected device.

Consequently, our method achieves accurate real-time detection of spy cameras with minimal computational resources. It effectively distinguishes between environments with and without hidden cameras, demonstrating robust and reliable detection performance.

We summarize our main contributions as follows:

- 1) Our method does not require specialized devices designed for a specific purpose but rather leverages readily available devices such as laptops or Raspberry Pis with WiFi monitoring capabilities.
- 2) Our approach does not require any specific action to trigger detection. Instead, it enables passive detection by continuously analyzing network traffic.
- 3) Our method eliminates the need for extensive data collection for training, thus offering a practical and simple approach to hidden spy camera detection adaptable to diverse environments and camera configurations.
- 4) Our method leverages the Nilsimsa algorithm to analyze packet similarity, making it effective regardless of encryption or protocol used by the camera.
- 5) Our approach leverages RSSI values to estimate the approximate location of detected spy cameras with accuracy to within 30 cm.

The remainder of the paper is organized as follows: Section II discusses the related work. Section III provides detailed insights into the methodology, covering data collection and preprocessing, the spy camera detection method utilizing data analysis procedures with the Nilsimsa algorithm, and the approach for detecting spy camera locations using

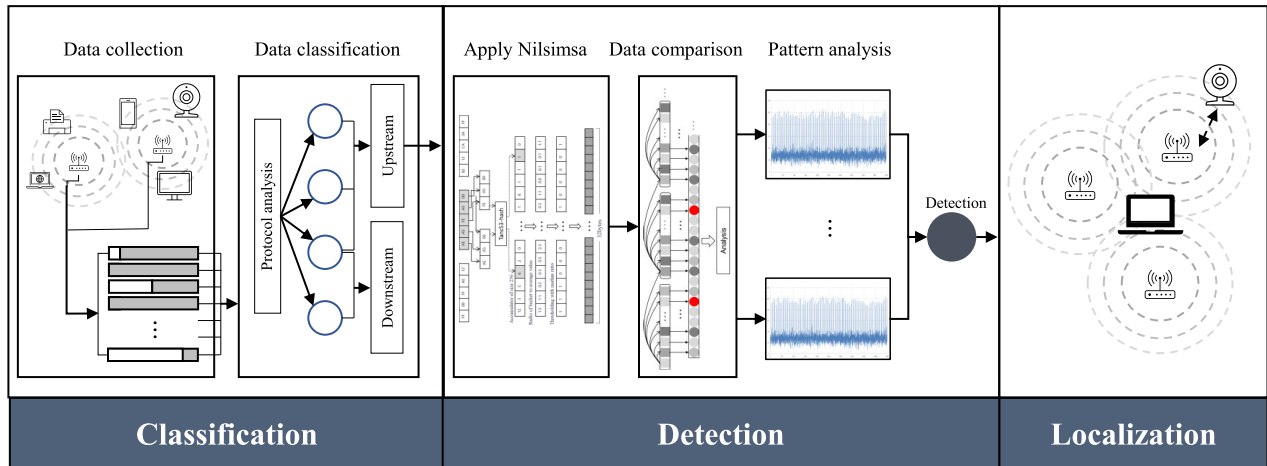


FIGURE 1. System architecture and framework.

WiFi RSSI. Section IV presents and discusses experimental results. Finally, Section V concludes the paper and outlines directions for future research.

II. RELATED WORK

As technology advances, the evolution of IP camera technologies has unfortunately led to their illegal misuse, infringing upon privacy and aiding criminal activities [1]. Numerous endeavors have been undertaken to devise techniques for detecting IP spy cameras that employ networks for video transmission and recording. These detection methods can be broadly categorized into hardware and software approaches.

Using hardware devices for detecting spy cameras presents several challenges, including their typically high cost, bulky size, and limited capability to detect specific analog streaming [1], [4]. Consequently, there has been a growing emphasis on software-based research, which has been more actively advancing in the field of spy camera detection. Table 1 presents a comparison of selected software techniques utilized for detecting spy cameras. Software-based techniques are broadly categorized into three main methods: CSI analysis, ML techniques, and traffic pattern analysis.

CSI can be used as a method to identify spy camera locations by analyzing WiFi traffic and the related signal metrics. CSI refers to a detailed set of physical information that measures the channel state between a transmitter and a receiver. Typically, CSI is obtained by analyzing the received signals at the receiver side [5]. CSI encompasses a comprehensive collection of physical data that assesses the channel condition between a transmitter and receiver. Typically, CSI is derived by analyzing the signals received on the receiver side. CSI can offer high accuracy in detecting hidden cameras due to the unique information provided by the channel state data [5].

CSI is typically combined with other technologies to improve its accuracy and performance. Muhammad Salman et al. introduced an approach for detecting spy cameras that leverages CSI data and bitrate fluctuations

observed during user movement. However, this method has a limitation, as it requires motion and a correlation between bitrate and CSI fluctuation to be present [1]. R. Cunningham employs signal metrics, encompassing RSSI, CSI, and Power Delay Profile (PDP), to ascertain the direction of received WiFi signals and estimate the distance to concealed WiFi sources. By comparing the variations in network traffic associated with human movement, these methods enable the identification of potential spy cameras [5]. Therefore, specific actions or movements are necessary to effectively initiate the detection process. Yongqiang Ma et al. propose a channel state information (CSI)-based indoor positioning technique called LENSER. LENSER uses a Linux 802.11n CSI tool to roam to APs and obtain CSI information. Based on this information, LENSER then estimates the location of indoor malicious devices using RSSI [13]. CSI data analysis and interpretation require specialized knowledge and sophisticated software, making it difficult for non-technical users to use. Additionally, CSI data analysis is performed based on human movement, so it can be difficult to detect spy cameras in locations without movement.

Another approach involves the utilization of ML techniques. By training ML neural networks, significantly improved results can be achieved. Dinhnguyen Dao et al. employ a deep learning model that combines the CNN-BiLSTM structure. This model analyzes CSI data to detect human motion [4]. The machine learning approach requires a large amount of labeled data to train the model, which can be challenging to obtain. Additionally, CSI traffic data can be noisy due to phenomena such as packet jitter and loss, which makes labeling difficult. As a result, the results can vary significantly depending on the amount or quality of the data.

Some methods employ ML techniques and RSSI to determine the location of spy cameras [14], [15]. The method of detecting location using the smartphone's ToF (Time-of-Flight) sensor to detect the position has the advantage of being easy to use because it utilizes sensors that exist on ordinary

smartphones. However, it has the limitation of having to scan each one at the ideal distance [14].

There are also methods for detecting spy cameras through ML and traffic patterns [6], [7], [16]. Wu et al. introduce a similarity analysis technique examining the timing characteristics of recorded video and network traffic from WiFi cameras [6]. However, a challenge with this method is that it requires simultaneous video recordings for analysis, and a significant number of data samples must be collected for neural network training. MotionComp uses an Android application to detect the location of cameras using a new wireless camera localization technology. However, this approach has two limitations: it relies on MAC addresses from device manufacturer organizations to find wireless camera traffic, and it only works on cameras that provide motion detection features [7]. FindSpy uses a large amount of unlabeled data for pre-training, and then uses a NN-LSTM classifier to detect wireless cameras. Although it can be trained with relatively less data than other machine learning methods, it has the limitation of requiring a large amount of data for training and does not provide any mechanism to identify the exact locations of spy cameras [16].

In addition, the method of detecting spy cameras through traffic analysis is effective in detecting wireless camera traffic in limited environments, but this method requires specific user movements or tasks. Additionally, it takes a long time to determine the camera location because it is a repetitive approach that requires user movement attempts for location and direction [8], [9], [10], [11], [12].

Our project aims to develop a novel method that enables rapid and efficient packet processing for spy camera detection, eliminating the need for pre-collected videos or data packets, as well as specialized hardware or equipment for CSI detection. We leverage the Nilsimsa algorithm to analyze packet similarity and employ RSSI signal extraction using standard network LAN cards that support commonly used and affordable wireless monitoring modes. This combination allows us to detect the location of cameras without relying on specialized tools.

III. PROPOSED METHOD

The primary objective of our research is to identify spy cameras transmitting data packets over the Internet in real-time, regardless of packet encryption, and to determine their approximate locations. Fig. 1 illustrates the proposed entire system architecture and framework for spy camera identification and positioning. The architecture consists of three main phases, they are data classification, detection, and localization. In the data classification phase, packets received from the Access Point (AP) are classified. Subsequently, Nilsimsa is employed for data analysis, utilizing Nilsimsa-generated data for comparative analysis to detect spy cameras. The pattern analysis part utilizes the analysis results to determine the presence of spy cameras. Finally, the localization phase is responsible for identifying the approximate location associated with the detected device.

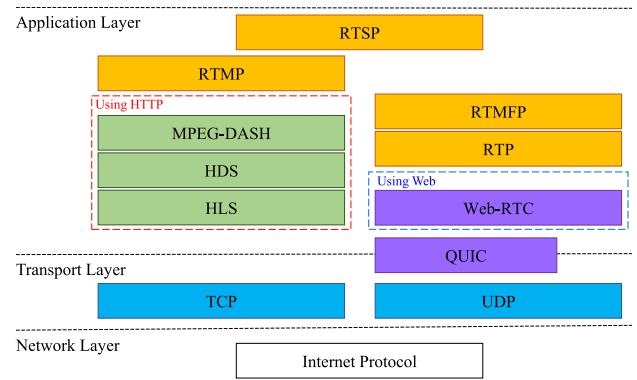


FIGURE 2. Multimedia and transport protocols within the protocol stack.

A. DATA CLASSIFICATION

The data classification phase comprises two essential parts: data collection, which gathers packets, and data classification, which involves categorizing and grouping the collected data based on the protocol.

The key function of the data collection part is to determine which packet data from each AP is brought into the protocol analysis part. IP camera transmits video through streaming [1]. Streaming refers to a technique utilized for the real-time or prerecorded playback of audio or video files directly from a streaming server [3]. There are three approaches to transmitting video streaming to end users: client-server, IP multicast, and peer-to-peer overlay [17]. IP multicast-based video transmission over the Internet is not widely adopted due to the need for transport routers that support multicast along the entire path [18]. Ensuring reliability in peer-to-peer systems involves significant challenges owing to the dynamic nature of peers and the breakage of overlay links [17]. The client-server model is overloaded on the server and is not scalable, but it is popular due to the protocols currently used in IP camera streaming [17], [19].

The placement of protocols used for each stream within the protocol stack is depicted in Fig. 2 [20]. Several protocols operate over the Transmission Control Protocol (TCP), including Real-Time Messaging Protocol (RTMP), Dynamic Adaptive Streaming over HTTP (MPEG-DASH), HTTP Dynamic Streaming (HDS), and HTTP Live Streaming (HLS) Real Time Streaming Protocol (RTSP). Additionally, Real-Time Media Flow Protocol (RTMFP), Real-Time Transport Protocol (RTP), Web Real-Time Communication (WebRTC), and Quick UDP Internet Connections (QUIC), employs the User Datagram Protocol (UDP).

RTMP is a protocol that employs TCP to concurrently transmit video, audio, and data among peers. TCP effectively mitigates packet loss and other network-related problems, thereby ensuring a relatively stable and reliable communication environment. This feature is especially valuable for wearable devices that frequently function in challenging communication conditions [20], [21].

RTSP is a protocol designed to establish and manage the delivery of media streaming data, including on-demand video

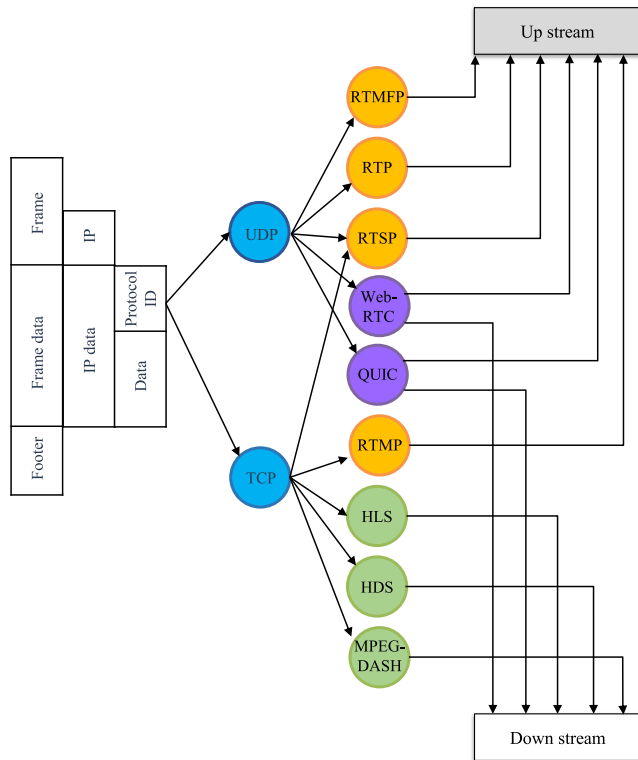


FIGURE 3. Protocol analysis.

or live audio streaming. It allows clients to request multiple audio and video streams that are delivered in a synchronized manner from servers to clients. RTSP functions as a network remote control for multimedia servers, enabling the real-time transmission of data over IP networks. It supports multiple transmission connections and offers a mechanism for selecting transmission channels while utilizing the RTP-based transmission approach [20], [22]. It mainly operates over TCP, but sometimes over UDP.

RTP is a network transport protocol designed for the delivery of real-time data, such as interactive video and audio, over multicast or unicast networks. It often operates in conjunction with RTSP in streaming media systems. RTP primarily relies on UDP for its communication and places a strong emphasis on the efficient transmission of real-time data. However, it does not manage resource reservation or ensure real-time Quality of Service (QoS) [20], [22].

RTMFP is a general-purpose endpoint-to-endpoint protocol designed for real-time transport of video, audio, and data. It operates on top of the UDP as its underlying transport protocol. RTMFP offers several important features that make it well-suited for real-time communication. It supports independent parallel message flows, enabling the simultaneous transmission of multiple data streams. This is particularly valuable for applications that need to transport various types of media or data concurrently [20].

MPEG-DASH is a streaming protocol designed to deliver MPEG media over HTTP. Streams are segmented into small

pieces and transmitted to receiving devices for continuous playback. MPEG-DASH divides multimedia content into brief segments, usually a few seconds long, with each segment encoded at different bitrates. These various bitrate representations are then hosted on a server [20], [23].

HDS is a streaming protocol developed by Adobe Systems for video streaming purposes. HDS is an HTTP-based streaming protocol used for streaming and playback of videos using Adobe Flash Player [24].

HLS is a bitrate live streaming technology introduced by Apple Inc. in 2009. It utilizes HTTP as the underlying protocol for streaming media content. HLS is widely supported by media players, internet browsers, mobile devices, and streaming media servers [25].

Web-RTC is an open-source project that enables real-time communication within web browsers and mobile applications through easy-to-use APIs. It eliminates the need for plugins or native application downloads by allowing direct peer-to-peer communication. With WebRTC, audio and video communication can seamlessly take place within web pages [26].

QUIC is a transport protocol designed to improve the performance of web applications and enhance the user experience. Developed by Google, QUIC operates over UDP instead of TCP, providing faster and more efficient data transmission [20], [27].

While several streaming protocols are available for various streaming purposes, our specific focus lies in upstream streaming of IP cameras over the Internet. Consequently, it becomes essential to classify these protocols into upstream and downstream categories, as depicted in Fig. 3. To distinguish between these categories, a close examination of the packet header becomes necessary. In this study, we harnessed the capability to analyze Ethernet frames via LAN (Local Area Network) connections or wireless frames through WiFi. LAN connections rely on wired communication, facilitating data exchange using Ethernet frames. In contrast, WiFi connections employ wireless communication technology for transmitting data in the form of wireless frames.

In the data classification phase illustrated in Fig. 1, the collected packet headers undergo sequential analysis. Initially, IPv4 (IP Version 4, 0×0800) packets are extracted by inspecting the Ethernet type field within the Ethernet frame [28]. The MAC destination address and MAC source address fields in the Ethernet header serve as identifiers for categorizing packets based on their source and destination devices. Likewise, when dealing with data received via an 802.11n wireless network, one can extract information from the MAC destination address and MAC source address fields while revealing encapsulated IP packets. By comparing and grouping packets that share identical MAC destination and MAC source addresses, they can be organized into discrete communication flows or connections. Subsequently, the Protocol ID field of the grouped packets is examined, importing only UDP (17) and TCP (6) packets, as these are

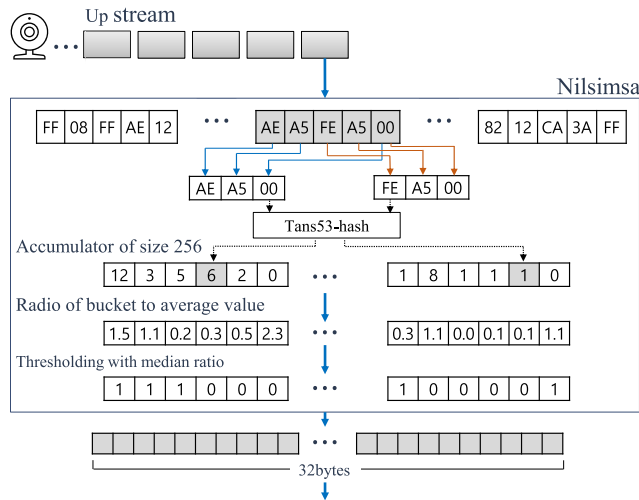


FIGURE 4. Nilsimsa algorithm.

the most commonly used protocols for signaling [28], [29], [30], [31], [32]. Further examination involves inspecting the payload segment of the packet and classifying it according to the streaming protocol. From these groupings, the packet group with the highest cumulative packet size is selected for transmission to the detection phase.

B. SPY CAMERA DETECTION

The detection phase is divided into three major parts: the utilization of a digest algorithm called Nilsimsa and the comparison and analysis of data based on the resulting value.

The Nilsimsa digest algorithm is employed to generate traffic signatures within network flows. This algorithm stands out due to its reduced sensitivity to minor input variations and its ability to produce similar hash values for similar inputs, which sets it apart from cryptographic hash algorithms. Unlike cryptographic hash algorithms, which generate entirely different hash values even for slight input changes, Nilsimsa maintains the property of generating similar hash values for comparable inputs [33]. Therefore, we chose Nilsimsa as it aligns with our objective of extracting similar packets.

As depicted in Fig. 4, Nilsimsa sequentially processes the collected packets in the following order [33]. It operates by reading the entire packet one byte at a time and uses a window size of five characters. The algorithm generates trigrams associated with the window and feeds each trigram into the **tran53 hash** function, resulting in a value ranging from 0 to 255. The value of the accumulator, which has a size of 256, is incremented by 1. After processing the entire input file, the accumulator stores the frequencies of the trigrams. In the subsequent step, the ratio for each bucket is calculated by averaging all the stored accumulators. Finally, the Nilsimsa algorithm generates a 32-byte hash by setting the i -th bit of the code to 1 if the i -th ratio is higher than the median of the ratios; otherwise, it is set to 0.

In the Data Comparison part, each packet is compared using the hash value generated by the Nilsimsa algorithm. The packet comparison method sequentially compares P_s packets in groups of a predetermined size, starting from the initial received packet. This sequential process is subsequently repeated for successive packets. To determine if two packets are the same, the corresponding hash values are compared to count the number of matching bits at the same position with the same value. In other words, the digest is bit-sampled using the **Hamming distance** as a measure of similarity. The similarity score is calculated by subtracting 128 from the count of matching bits within the hashes [34]. If two similarity scores are exactly equal, they have a value of 128; otherwise, if they are not exactly identical, the value is 0. These similarity scores are stored in an array of fixed size and then passed to the next pattern analysis part.

In the pattern analysis part, the system counts and tracks the number of values in the transmitted data from the previous stage that exceed the similarity score threshold. If these threshold-exceeding values display a consistent pattern of repetition or if a specific number of packets exceed the threshold within a designated timeframe, it is deduced that a surveillance camera is present.

C. LOCALIZATION

In the localization phase, if a spy camera is detected in the preceding detection phase, this phase is dedicated to estimating the approximate location of the spy cam. To utilize WiFi signals for approximating the camera's location, it's essential to configure the network interface to scan and gather nearby WiFi signals.

The IEEE 802.11 standard outlines two network discovery modes for mobile devices: passive scanning and active probing. In passive mode, access points broadcast beacons, and mobile devices listen passively for these beacons. In active probing mode, mobile devices continually send probe requests to reestablish connections with previously associated networks. These probe request packets contain the mobile device's MAC address in clear text, facilitating the constant broadcasting of presence and identification [35].

Furthermore, the RSSI data embedded within the periodically transmitted WiFi signals from APs offers the potential for mobile device localization. WiFi indoor localization primarily relies on RSSI data. However, accurately pinpointing the exact location in practical environments poses challenges because of signal interference effects caused by phenomena like signal reflection, diffraction, and shadowing on the received mobile terminal's RSSI. Nevertheless, in most cases, the data shows fluctuations within an acceptable range, closely resembling the actual values [36]. Therefore, determining the approximate location is generally feasible.

The localization of the spy camera is performed in the following sequence. A WiFi scanning procedure is conducted to capture the transmitted signal of the device identified using the MAC address obtained in the previous step. During this

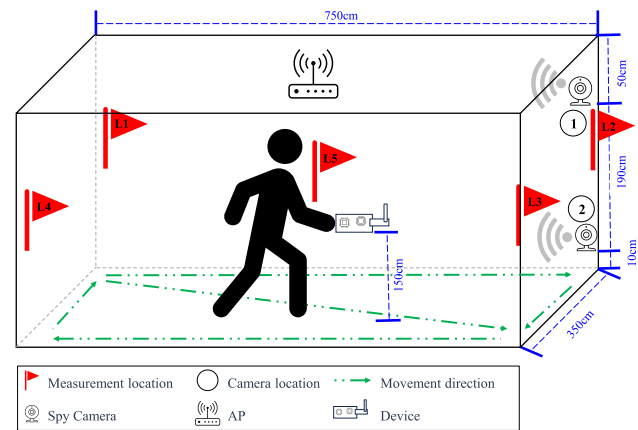


FIGURE 5. Activities to find spy cameras via RSSI.

scanning procedure, RSSI values are collected to quantify the signal strength of the corresponding WiFi device. Elevated signal strength at a specific location within the collected set of RSSI values indicates the potential presence of the spy cam. The identified RSSI values are then mapped to their respective locations, facilitating the identification of areas where spy cameras may be concealed.

IV. PERFORMANCE EVALUATION

This section presents and discusses the test-bed setup and the results obtained from the conducted experiments.

A. EXPERIMENT ENVIRONMENT

Our experimental setup is illustrated in Fig. 5. Initially, we constructed a laboratory environment in a rectangular shape measuring 350cm × 750cm. In one corner, we positioned the camera. The testbed includes the following components: an AP, a vstarcam 100a wireless IP camera, an Android smartphone with the vstarcam application installed, an EFM ipTime N150UA USB 2.0 wireless LAN card, and a Raspberry Pi 3 model B+ for RSSI collection. A Linux Ubuntu desktop or laptop can be used for traffic sniffing/monitoring, but a Raspberry Pi can also be used for this purpose.

The APs employed in our experimental setup were affordable, entry-level off-the-shelf devices specifically designed for the average home user.

B. IP CAMERA SETTING

The vstarcam 100a IP camera is located at the corner of Fig. 5. A detailed specification of the vstarcam camera is outlined in Table 2. To establish the camera’s connection with the AP, we followed the instructions provided in the vstarcam application on the mobile phone. The camera was connected to the AP following the order specified in the app, and we took the necessary steps to ensure accurate camera configuration. Subsequently, we verified the successful transmission of video footage within the application.

TABLE 2. IP camera specifications.

Category	Specification
Digital Format	Multistream 720P / VGA /QVGA 30FPS
Protocol	ONMF 2.0, RTSP, HTTP, SMTP
Lens	3.6mm
Power consumption	2.5W
Horizontal View Angle	270°
Vertical View Angle	80°

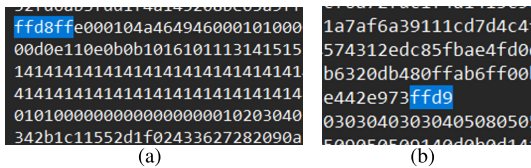


FIGURE 6. JPEG markers: (a) The JPEG Start Of Image (SOI) marker and (b) The End Of Image (EOI) marker.

C. IP CAMERA PACKET CAPTURE

Following the configuration of the IP camera, we captured traffic from both wired and wireless connections, as the camera supports both modes. We also ran different applications, such as web browsing and YouTube video playback, to simulate a real-world environment. The collected traffic was subjected to analysis, involving classification based on source and destination MAC addresses, and grouping based on cumulative data size over 10 minutes. As expected, data generated by the camera exhibited the largest data size. Furthermore, our analysis revealed that JPEG/MPEG image and video files were transmitted without encryption [37]. JPEG data appeared as a sequential series of blocks, with each block characterized by marker values, commonly referred to as JPEG Magic Numbers. By examining the payload of the captured data in Fig. 6, we confirmed the presence of JPEG markers, specifically the Start of Image (SOI) marker FF D8 FF and the End of Image (EOI) marker FF D9 [37]. It’s important to note that this data collection process was performed regardless of whether the data was encrypted.

The IP camera provided outputs VGA-format images with dimensions of 640 × 360 pixels and a color depth of 3 (RGB). Due to Ethernet’s maximum transmission unit (MTU) size of 1500 bytes, each image is segmented into multiple packets for transmission. In this scenario, approximately 460 images are divided into packets. These packets are processed through Nilsimsa in the Detection phase, as depicted in Fig. 1, and then forwarded to the data comparison part. In the data comparison part, the packets are compared using a specific comparison set denoted as P_s. For our experiment, we reduced the storage size of 460 packets to 1/10, resulting in 46 packets used for analyzing similarity patterns. Subsequently, packet sets containing 100 and 500 packets of varying sizes were employed for further comparison.

D. PACKET ANALYSIS

The results are depicted in Fig. 7, with Fig. 7(a) and Fig. 7(b) illustrating the comparison of 46 packets, Fig. 7(c)

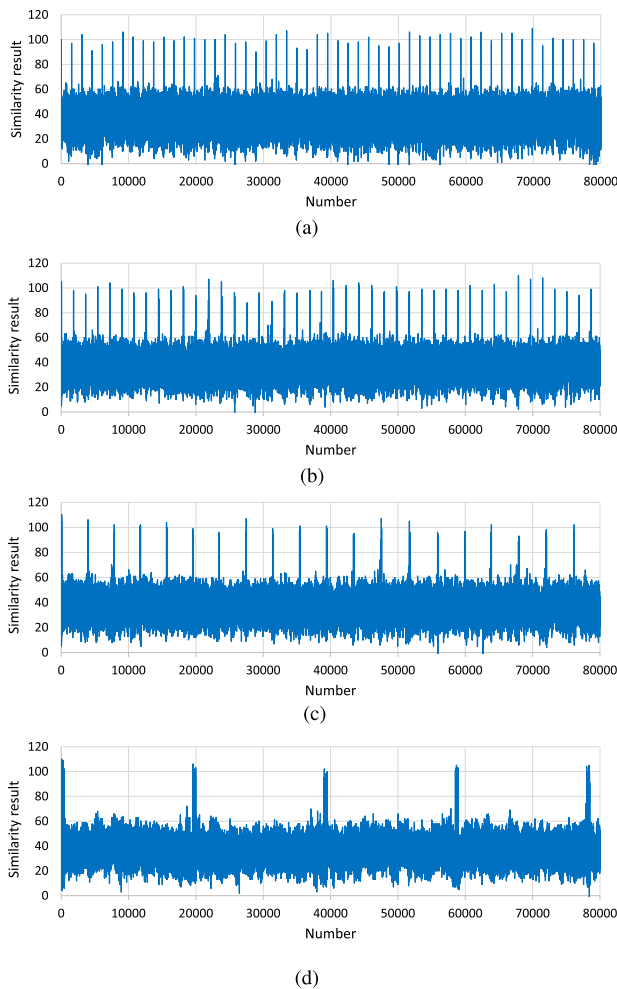


FIGURE 7. The results according to data comparison size P_s : (a) Comparison size in wired connection (46) and (b) Comparison size in wireless connection (46) and (c) Comparison size (100) and (d) Comparison size (500).

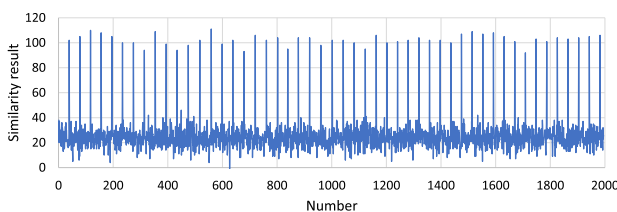


FIGURE 8. Comparison with a single value.

showcasing the comparison of 100 packets, and Fig. 7(d) displaying the comparison of 500 packets. Furthermore, Fig. 7(a) represents the captured packets from a wired connection, while Fig. 7(b) shows the packets captured from a wireless connection. The comparison of these results reveals a repetition of similar patterns in both wired and wireless packets. From the four graphs, it is evident that variations in packet size mainly influence the interval at which similar packets are identified, with minimal impact on the periodicity of similarity patterns.

Additionally, when the comparison size is small, an examination of whether the high similarity value results from

similarities between adjacent image areas is conducted by comparing the initially detected packet with subsequent packets. The results of this comparison can be observed in Figure 8, and they align with the outcomes of the prior tests. Based on these results, we have determined that utilizing 46 packets as the comparison size is optimal.

To assess the influence of packets on motion detection, we conducted tests comparing packet collection results under various motion conditions. Our objective was to gain insights into how packets relate to motion. We conducted four types of tests: packets when no spy cameras were present, streaming packets in the absence of motion, packets with minimal motion, and packets with significant motion occurring at the center of the frame. Considering the standard MTU size of modern networks, typically set to 1500 bytes, we focused on collecting packets that met the requirement of being sufficiently large ($frame.len \geq 1000$). These packets were specifically collected from the source IP address of the Vstarcam camera. We analyzed these packets using Nilsimsa, a hash-based similarity measurement technique, and subsequently, we conducted pattern analysis.

As depicted in Fig. 9, even in environments without surveillance cameras or where packet flows vary depending on activity, the similarity consistently remains below approximately 60 and never exceeds 70. Additionally, in Fig. 10, when capturing packets in another motionless environment, the similarity threshold of 70 is not exceeded when there are differences in packet similarity. This indicates that the analysis results obtained in the absence of the spy camera exhibit a distinct pattern or characteristic, allowing us to establish a baseline similarity score threshold.

By comparing the similarity scores of subsequent analysis result packets with this threshold value, we can effectively identify whether the spy camera is present or not. If the similarity score of a packet surpasses the threshold, it suggests the presence of the spy camera and triggers further investigation. Conversely, if the similarity score remains below the threshold, it can be concluded that the spy camera is not present.

The results obtained, as depicted in Fig. 9, reveal the outcomes of the analysis. Fig. 9(b) shows a still image, and Fig. 9(c) shows the result with minor motion. When comparing these two results, some changes are evident, but no significant differences are observed. However, in the case of Fig. 9(d), a distinct discrepancy was observed between the values recorded during significant movement and the values recorded before and after that particular moment. Nevertheless, even in cases where the similarity values are not high due to motion, there are instances where the values exceed the threshold (70). Therefore, it is evident that by inspecting packet similarity, it is possible to detect a spy camera even in the presence of significant movements.

E. COLLECT AND ANALYSIS RSSI VALUES

After detecting the presence of spy cameras in the previous step, we proceeded to measure WiFi RSSI values in the

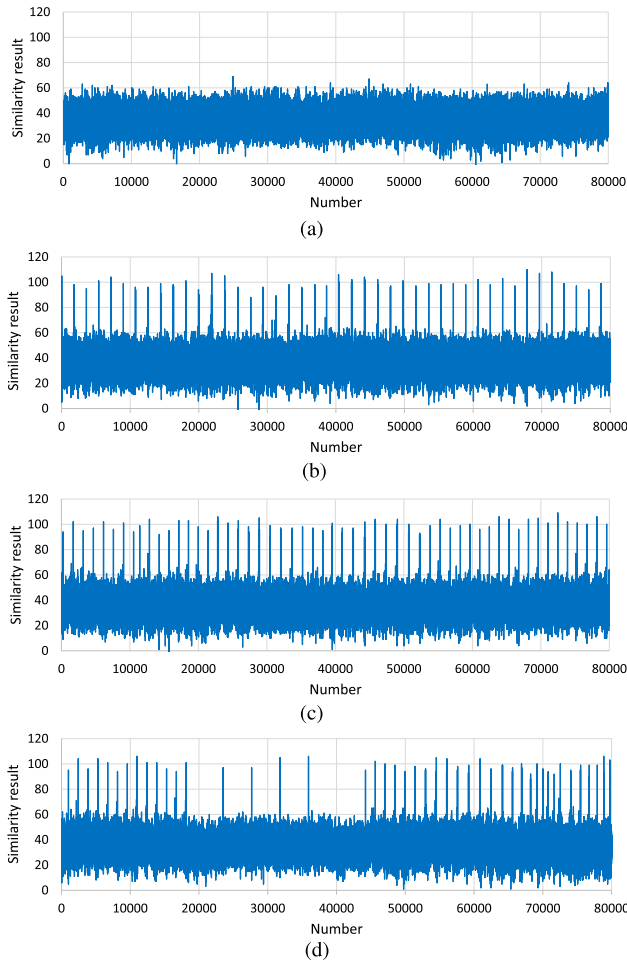


FIGURE 9. The comparison results according to movement: (a) No Spy camera and (b) Static and (c) Minor motion and (d) Significant motion.

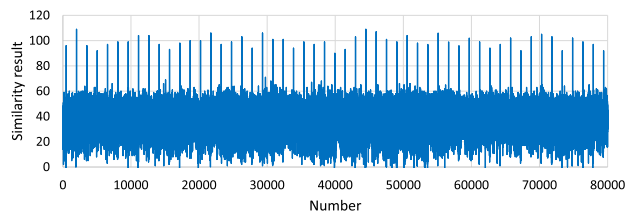


FIGURE 10. Static motion in another environment.

experimental environment depicted in Fig. 5 to determine the positions of the cameras. The camera's RSSI was measured twice: once from the bottom (Position 1 in Fig. 5, 10 cm above the floor) and once from the top (Position 2 in Fig. 5, 200 cm above the floor). We connected a USB wireless LAN card to the Raspberry Pi and configured the wireless LAN card to WiFi monitoring mode.

Next, we read the WiFi RSSI values using the MAC address of the camera obtained in the previous step. The experimental method involved conducting measurements at five different locations as indicated in Fig. 5. In other words, RSSI measurements were taken at each corner while moving along the corners of the rectangular area. Upon returning to the initial starting position, measurements were also taken at the center. The RSSI measurement position of the Raspberry

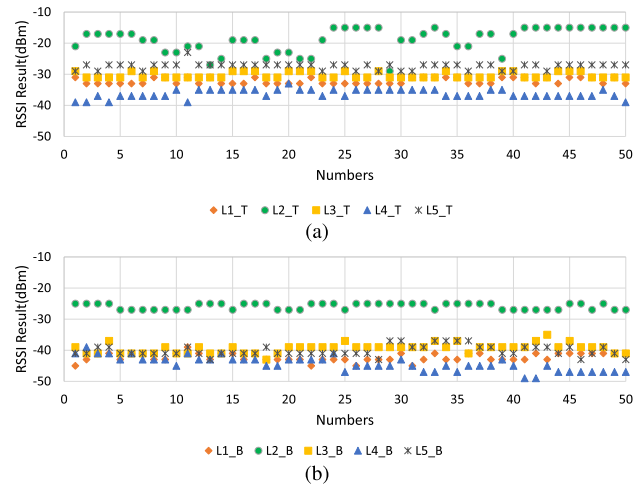


FIGURE 11. Collected WiFi RSSI Values: (a) Camera positioned at the top and (b) Camera positioned at the bottom.

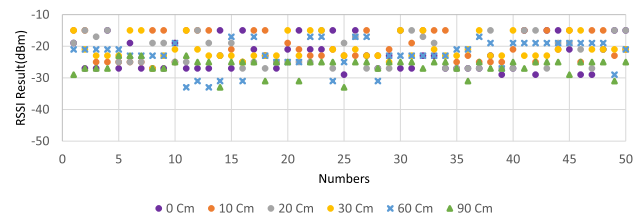


FIGURE 12. Variation of WiFi RSSI values with distance.

Pi was set at approximately 150 cm, corresponding to the chest height of a person carrying it.

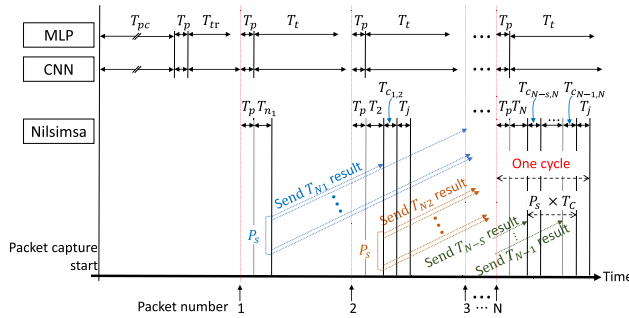
The WiFi RSSI values obtained at each respective location are presented in Fig. 11. Here, the measurement positions are denoted as L1, L2, L3, L4, and L5, with T representing the top position and B representing the bottom position. Analysis of the measurements taken at the five different locations in Fig. 5 reveals variations in the RSSI values based on the camera's position. In Fig. 11, it is evident that the RSSI values are not constant at each location and exhibit fluctuations in signal strength.

However, a notable trend emerges as the measurement position approaches the camera's location: RSSI values tend to be higher compared to values obtained at other positions. This trend is further highlighted when examining the average RSSI values at the respective positions, as shown in Table 3. Specifically, the RSSI values at L2, the closest position to the camera, and L4, the furthest position, exhibit a substantial difference of dBm or more. This consistent pattern holds true regardless of whether the camera is positioned above or below. These findings provide valuable insights into the relationship between the measurement position and the camera's impact on RSSI values. It suggests that the presence of the camera directly influences signal strength, resulting in higher RSSI values when measurements are taken in close proximity to the camera. Consequently, we can approximate the location of the spy camera based on these observations.

To further investigate the relationship between RSSI values and distance, we conducted experiments at various distances

TABLE 3. RSSI average value comparison by location (dBm).

	L1 (750 cm)	L2 (20 cm)	L3 (350 cm)	L4 (830 cm)	L5 (410 cm)
Top	-32.2	-18.8	-30.32	-36.24	-27.4
Bottom	-42.24	-25.88	-39.4	-44.32	-40.16

**FIGURE 13.** Timing diagram comparing camera detection methods.**TABLE 4.** Notations.

Item	Explanation
T_{pc}	Packet collection time
T_p	Packet preprocessing time
T_{tr}	Packet training time
T_{n1}	The processing time of the Nilsimsa algorithm for the 1st arrived packet
T_N	The processing time of the Nilsimsa algorithm for the Nth arrived packet
$T_{c1,2}$	The processing time for comparing the 1st packet and the 2nd packet
T_c	The processing time for comparing the N-1th packet and the Nth packet
T_j	The judgment time
P_s	The total size transmitted for packet comparison

(0, 10, 20, 30, 60, and 90 cm) with the camera positioned at a mid-point. As Fig. 12 shows, the RSSI values fluctuate within the error range for distances of 30 cm or less. Interestingly, the fluctuations are more pronounced when the measurement position is directly adjacent to the camera. However, a slight difference emerges in the RSSI results for distances of 60 and 90 cm, compared to distances of 30 cm or less. The average RSSI values are -20.6 dBm at 30 cm, -22.52 dBm at 60 cm, and -26.24 dBm at 90 cm, indicating a decrease in RSSI strength with increasing distance. Based on these findings, the camera's position can be estimated to be within approximately 30 cm. This distance is close enough to detect the camera. Therefore, our findings suggest that the camera can be reliably detected within this proximity.

F. COMPARISON WITH DIFFERENT METHODS

In this section, we conduct a performance comparison between our proposed method and machine learning models, focusing on metrics such as accuracy (ACC), true positive rate (TPR), true negative rate (TNR), and computation timing. Specifically, we compare our method with two machine learning models: MLP (Multi-Layer Perceptron) and CNN (Convolutional Neural Network). Classical MLP models tend to exhibit strong performance when the number of input packets, representing the input dimension, is relatively small. However, when dealing with high-dimensional inputs, CNNs have been reported to deliver higher accuracy [38].

TABLE 5. The confusion matrix results for the initially conducted test.

N=60000	Predicted (MLP)		Predicted (CNN)		Predicted (Nilsimsa)	
	Yes	No	Yes	No	Yes	No
Actual	Yes	TP = 10,715 FN = 535	TP = 9,661 FN = 1,589	TP = 8,877 FN = 36,123	TP = 8,877 FN = 36,123	TP = 8,877 FN = 36,123
	No	FP = 660 TN = 3,090	FP = 2,279 TN = 1,471	FP = 0 TN = 15,000	FP = 0 TN = 15,000	FP = 0 TN = 15,000

The input data for the MLP model is in a 2D matrix format, eliminating the need to reshape the dataset. The MLP model under examination comprises an initial input layer followed by seven dense layers. Each of these layers employs a ReLU (Rectified Linear Unit) activation function, with the output of one layer serving as the input for the next. The final dense layer utilizes a sigmoid activation function to produce output for the output layer.

On the contrary, the CNN deep learning model is composed of convolutional layers, max-pooling layers, and dense layers. It includes two convolutional layers with ReLU activation functions, separated by a MaxPooling layer. These convolutional layers employ 5×64 kernels, and the model incorporates a total of four dense layers. The features extracted by the CNN's feature extractor component are then fed as input to the dense layers responsible for traffic classification. Prior to being passed to the dense layers, a flatten layer is applied to the extracted features. Additionally, the final dense layer employs the sigmoid activation function.

For the purpose of comparison, we collected packet data over a duration of approximately 40 minutes. During this time, we allocated approximately 10 minutes for collecting packets in a scenario where no camera was present. Consequently, we amassed approximately 60,000 packets, all with sizes exceeding 1000 bytes. Of these, 25% were selected as test data for the machine learning models. Additionally, to ensure a fair comparison, all models were trained on the same packet dataset. In the construction of the MLP and CNN models, preprocessing the collected packet data is a crucial step. The packet data is defined to possess a fixed size of 1500 bytes. If the packet size falls below this threshold, it is padded with zeros, whereas if it exceeds 1500 bytes, it is truncated to fit the specified size.

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$TPR = \frac{TP}{TP + FN} \quad (2)$$

$$TNR = \frac{TN}{TN + FP} \quad (3)$$

A confusion matrix is a valuable performance evaluation tool in machine learning, providing insights into the accuracy of a classification model. In our analysis, we utilized the confusion matrix to extract essential metrics, including true positives (TP), false negatives (FN), true negatives (TN), and false positives (FP). The corresponding test results can be found in Table 5. Based on these results, the following

formulas were used to calculate ACC, TPR, and TNR [39]. Based on the results, the MLP model achieved an ACC of 0.92, a TPR of 0.952, and a TNR of 0.824. In contrast, the CNN model yielded an ACC of 0.742, a TPR of 0.858, and a TNR of 0.392. As for the Nilsimsa method, the ACC was 0.397, the TPR was 0.197, and the TNR was 1. However, evaluating the accuracy of Nilsimsa solely based on the number of occurrences in the overall packet results can be problematic. Nilsimsa, by default, considers the presence of spy cameras when a certain number of values exceed the threshold. Therefore, the number of TP is not the primary concern. The crucial aspect lies in the TNR results, which are close to 1, indicating that it accurately recognizes situations where there are no spy cameras.

Additionally, to facilitate the performance comparison, we measured the data collection time, training time, and testing time. Fig. 13 provides an illustration of the testing procedure and timing diagram, while Table 4 offers an explanation of the notations used in Fig. 13. As shown in Fig. 13, our method eliminates the need for packet data collection time or training time. Upon the arrival of each packet, the Nilsimsa algorithm is immediately applied, and the result is stored. Subsequently, when the next packet arrives, the Nilsimsa algorithm is applied to that packet, and the comparison between the two packets is performed promptly. Based on the threshold value, the system can rapidly determine whether the packet contains spy camera data.

In the context of ML, the packet collection process spans 40 minutes, and the preprocessing of 60,000 packets requires 4.934 seconds. For the MLP model, training takes 224.758 seconds, and the testing time for each packet is 0.045 seconds. On the other hand, for the CNN model, training lasts 305.1 seconds, and the testing time for each packet is 0.051 seconds. In stark contrast, Nilsimsa exhibits significantly lower time requirements. The protocol analysis for the packet header during data input takes only 0.0001 seconds, the Nilsimsa execution time is 0.006 seconds, data comparison consumes 0.00001 seconds, and the threshold comparison time is 0.00001 seconds. Consequently, the Nilsimsa method demonstrates a remarkably reduced processing time.

Table 6 displays the results obtained by testing packets collected under a modified camera position within the same laboratory environment using the previously trained models. For the MLP model, the ACC value is 0.655, TPR is 0.649, and TNR is 0.757. In contrast, the CNN model achieves an ACC of 0.678, TPR of 0.858, and TNR of 0.695. Nilsimsa consistently attains a TNR value of 1. When the packet collection environment is altered, the confusion matrix results for machine learning models, except for CNN's TPR, notably decrease. Conversely, our method demonstrates the capability to accurately identify environments without spy cameras, with results influenced by the chosen threshold value.

Certainly, increasing the packet data collection time to accumulate a substantial amount of data can enhance

TABLE 6. The confusion matrix results for the collected packets under the changed packet collection environment.

N=60000		Predicted (MLP)		Predicted (CNN)		Predicted (Nilsimsa)	
		Yes	No	Yes	No	Yes	No
Actual	Yes	TP = 1297	FN = 699	TP = 1389	FN = 607	TP = 76	FN = 1920
	No	FP = 26	TN = 81	FP = 70	TN = 37	FP = 0	TN = 107

the accuracy of machine learning models. However, this approach comes with the trade-off of longer data collection and training times. Moreover, it introduces uncertainty regarding the maintenance of accuracy in changing environments.

V. CONCLUSION AND FUTURE WORK

In this paper, we introduce an innovative approach to detect the presence and approximate location of spy cameras within 30 cm without relying on predefined behavior or extensive training. Our method leverages Nilsimsa, a hash function known for generating similar hash values for similar inputs, to assess the similarity between sequentially received packets using the Hamming distance. Based on our analysis of similarity results, we set a threshold of 70 for identifying potential spy cameras in the network. This threshold setting is reasonable because similarity consistently remains below this threshold in tests involving a variety of movements and environments. If any packets exhibit a similarity result exceeding this threshold, it signals the potential presence of spy cameras. Furthermore, by utilizing off-the-shelf devices and analyzing the corresponding RSSI values, our approach offers a practical and accessible means to estimate the approximate location of the camera seamlessly and naturally.

As a result, our proposed method integrates Nilsimsa-based similarity analysis with WiFi RSSI value exploitation, offering an effective and comprehensive approach for real-time spy camera detection without the need for predefined behaviors or extensive training.

For our future study, we aim to conduct a comprehensive comparative analysis of various streaming protocols that have not been addressed in this research.

REFERENCES

- [1] M. Salman, N. Dao, U. Lee, and Y. Noh, "CSI: DeSpy: Enabling effortless spy camera detection via passive sensing of user activities and bitrate variations," *Proc. ACM Interact. Mobile, Wearable Ubiquitous Technol.*, vol. 6, no. 2, pp. 1–27, Jul. 2022.
- [2] P. A. Abdalla and C. Varol, "Testing IoT security: The case study of an IP camera," in *Proc. 8th Int. Symp. Digit. Forensics Secur. (ISDFS)*, Jun. 2020, pp. 1–5.
- [3] A. Nurrohman and M. Abdurrohmam, "High performance streaming based on H264 and real time messaging protocol (RTMP)," in *Proc. 6th Int. Conf. Inf. Commun. Technol. (ICOICT)*, May 2018, pp. 174–177.
- [4] D. Dao, M. Salman, and Y. Noh, "DeepDeSpy: A deep learning-based wireless spy camera detection system," *IEEE Access*, vol. 9, pp. 145486–145497, 2021.
- [5] R. Cunningham and W. L. Tan, "Detection and localization of hidden Wi-Fi cameras," in *Proc. 27th Asia-Pacific Conf. Commun. (APCC)*, Oct. 2022, pp. 12–17.

- [6] K. Wu and B. Lagesse, "Do you see what I see? Detecting hidden streaming cameras through similarity of simultaneous observation," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. (PerCom)*, Mar. 2019, pp. 1–10.
- [7] Y. He, Q. He, S. Fang, and Y. Liu, "MotionCompass: Pinpointing wireless camera via motion-activated traffic," in *Proc. 19th Annu. Int. Conf. Mobile Syst. Appl. Services*, New York, NY, USA, Jun. 2021, pp. 215–227.
- [8] Y. Cheng, X. Ji, T. Lu, and W. Xu, "Dewicam: Detecting hidden wireless cameras via smartphones," in *Proc. Asia Conf. Comput. Commun. Secur.*, May 2018, pp. 1–13.
- [9] Y. Cheng, X. Ji, T. Lu, and W. Xu, "On detecting hidden wireless cameras: A traffic pattern-based approach," *IEEE Trans. Mobile Comput.*, vol. 19, no. 4, pp. 907–921, Apr. 2020.
- [10] J. Heo, S. Gil, Y. Jung, J. Kim, D. Kim, W. Park, Y. Kim, K. G. Shin, and C.-H. Lee, "Are there wireless hidden cameras spying on me?" in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, New York, NY, USA, Dec. 2022, pp. 714–726.
- [11] T. Liu, Z. Liu, J. Huang, R. Tan, and Z. Tan, "Detecting wireless spy cameras via stimulating and probing," in *Proc. 16th Annu. Int. Conf. Mobile Syst. Appl. Services*, New York, NY, USA, Jun. 2018, pp. 243–255.
- [12] Singh and A. Deep, "I always feel like somebody's sensing me: A framework to detect, identify, and localize clandestine wireless sensors," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, May 2021, pp. 1829–1846.
- [13] Y. Ma, X. Luo, R. Li, S. Du, and W. Liu, "LENSER: A channel state information based indoor localization scheme for malicious devices," in *Proc. IEEE 20th Int. Conf. Mobile Ad Hoc Smart Syst. (MASS)*, Sep. 2023, pp. 461–470.
- [14] S. Sami, S. R. X. Tan, B. Sun, and J. Han, "LAPD: Hidden spy camera detection using smartphone time-of-flight sensors," in *Proc. 19th ACM Conf. Embedded Netw. Sensor Syst.*, New York, NY, USA, Nov. 2021, pp. 288–301.
- [15] Z. Li, Z. Xiao, Y. Zhu, I. Pattarachanyakul, B. Y. Zhao, and H. Zheng, "Adversarial localization against wireless cameras," in *Proc. 19th Int. Workshop Mobile Comput. Syst. Appl.*, New York, NY, USA, Feb. 2018, pp. 87–92.
- [16] Z. Shi, H. Wu, J. Zhang, M. Zhang, and W. Huang, "FindSpy: A wireless camera detection system based on pre-trained transformers," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2023, pp. 816–822.
- [17] B. U. Maheswari and T. K. Ramesh, "An improved delay-resistant and reliable hybrid overlay for peer-to-peer video streaming in wired and wireless networks," *IEEE Access*, vol. 6, pp. 56539–56550, 2018.
- [18] J. Yang, E. Yang, Y. Ran, Y. Bi, and J. Wang, "Controllable multicast for adaptive scalable video streaming in software-defined networks," *IEEE Trans. Multimedia*, vol. 20, no. 5, pp. 1260–1274, May 2018.
- [19] M. Uitto and A. Heikkinen, "Evaluation of live video streaming performance for low latency use cases in 5G," in *Proc. Joint Eur. Conf. Netw. Commun. 6G Summit (EuCNC)*, Jun. 2021, pp. 431–436.
- [20] J. Cao, X. Su, B. Finley, A. Pauanne, M. Ammar, and P. Hui, "Evaluating multimedia protocols on 5G edge for mobile augmented reality," in *Proc. 17th Int. Conf. Mobility, Sens. Netw. (MSN)*, Dec. 2021, pp. 199–206.
- [21] Y. Jing and Q. Gao, "Design and implementation of live streaming system for wearable devices," in *Proc. IEEE 9th Int. Conf. Softw. Eng. Service Sci. (ICSESS)*, Nov. 2018, pp. 1–5.
- [22] Jianbing, Liang, and C. Shuhui, "The design and implementation of RTSP/RTP multimedia traffic identification algorithm," *J. Phys. Conf. Ser.*, vol. 1168, no. 5, Feb. 2019, Art. no. 052033.
- [23] I. Sodagar, "The MPEG-DASH standard for multimedia streaming over the internet," *IEEE Multimedia Mag.*, vol. 18, no. 4, pp. 62–67, Apr. 2011.
- [24] Adobe HTTP Dynamic Streaming (HDS). Accessed: 2011. [Online]. Available: <http://www.adobe.com/products/hds-dynamic-streaming.html> and <https://dl.acm.org/doi/pdf/10.1145/2490826>
- [25] M. Prasad and P. Venkateshwari, "Social educational streaming platform using HTML live streaming," in *Proc. Int. Mobile Embedded Technol. Conf. (MECON)*, Mar. 2022, pp. 226–229.
- [26] A. K. Pandey, S. Kumar, B. Rajendran, and B. S. Bindhumadhava, "E-parakh: Unsupervised online examination system," in *Proc. IEEE Region 10 Conf. (TENCON)*, Nov. 2020, pp. 667–671.
- [27] T. Shreedhar, R. Panda, S. Podanev, and V. Bajpai, "Evaluating QUIC performance over web, cloud storage, and video workloads," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1366–1381, Jun. 2022.
- [28] IEEE Standard for Ethernet, Standard IEEE 802.3-2018 (Revision of IEEE Std 802.3-2015), Jul. 2022, pp. 1–7025.
- [29] A. Das and S. Debbarma, "Performance of jumbo sized data on jumbo frame and Ethernet frame using UDP over IPv4/IPv6," in *Proc. 2nd Int. Conf. Adv. Comput., Netw. Secur.*, Dec. 2013, pp. 204–207.
- [30] J. Postel, *Internet Protocol*, document Rec. RFC 791, Sep. 1981.
- [31] J. Postel, *Transmission Control Protocol*, document Rec. RFC 793, Sep. 1981.
- [32] J. Postel, *User Datagram Protocol*, document Rec. RFC 768, Aug. 1980.
- [33] B. Charyyev and M. H. Gunes, "Locality-sensitive IoT network traffic fingerprinting for device identification," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1272–1281, Feb. 2021.
- [34] J. Oliver, C. Cheng, and Y. Chen, "TLSH—A locality sensitive hash," in *Proc. 4th Cybercrime Trustworthy Comput. Workshop*, Sydney, NSW, Australia, Nov. 2013, pp. 7–13.
- [35] Y. Li, J. Barthelemy, S. Sun, P. Perez, and B. Moran, "A case study of WiFi sniffing performance evaluation," *IEEE Access*, vol. 8, pp. 129224–129235, 2020.
- [36] P. Wang and Y. Luo, "Research on WiFi indoor location algorithm based on RSSI ranging," in *Proc. 4th Int. Conf. Inf. Sci. Control Eng. (ICISCE)*, Jul. 2017, pp. 1694–1698.
- [37] A. Tekeoglu and A. S. Tosun, "Investigating security and privacy of a cloud-based wireless IP camera: NetCam," in *Proc. 24th Int. Conf. Comput. Commun. Netw. (ICCCN)*, Las Vegas, NV, USA, Aug. 2015, pp. 1–6.
- [38] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [39] D. Valero-Carreras, J. Alcaraz, and M. Landete, "Comparing two SVM models through different metrics based on the confusion matrix," *Comput. Oper. Res.*, vol. 152, Apr. 2023, Art. no. 106131.



HYEYOUNG AN received the B.S. degree from the College of Computer Science, Chungnam National University, South Korea, in 2001, and the M.S. degree from the Department of Electrical and Computer Engineering, Chungbuk National University, in 2023. She is currently pursuing the Ph.D. degree with the Department of Computer Engineering, Chungbuk National University. Her research interests include networking technologies, sensing and data analytics, optimization, AI, and the IoT.



WOOJIN PARK (Student Member, IEEE) received the B.S. degree from the School of Computer Engineering, Chungbuk National University, South Korea, in 2023. He is currently pursuing the master's degree with the Department of Computer Engineering, Chungbuk National University.



SOOCHANG PARK (Member, IEEE) received the Ph.D. degree from Chungnam National University, South Korea, in 2011. He was with Rutgers University, USA, as a Postdoctoral Researcher, in 2012, and Institut Mines-Télécom, Telecom SudParis, France, as a Research Engineer, from 2013 to 2015. He was with The Hong Kong University of Science and Technology (HKUST), as a Research Associate, in 2016. Since 2017, he has been with Chungbuk National University (CBNU), South Korea, where he has been an Associate Professor with the Department of Computer Engineering, since 2021. His research interests include networking technologies, sensing and data analytics, and mobile and the Internet of Things-based smart applications.

• • •